

What is `__new__`?

- `__new__` is a special method in Python that's called before `__init__`.
 - It is responsible for creating a new instance of a class.
 - It's commonly used in advanced patterns like Singleton, Immutable objects, or metaclass-based logic.
-

What is a Class Decorator?

A class decorator is a function that takes a class as input and returns a modified or extended class.

What Does "Static Mode and New Class" Mean?

This typically refers to:

- **Static mode** → Using a decorator or `__new__` without relying on previously created instances (like a static factory or utility).
- **New class** → Dynamically creating or returning a wrapped version of the original class with modified behavior.



Example: Combining a Decorator and `__new__` to Modify Class Instantiation

python

📄 Copiar

✎ Editor

```
# Decorator that turns any class into a Singleton
def singleton(cls):
    class Wrapped(cls):
        _instance = None

        def __new__(cls, *args, **kwargs):
            if cls._instance is None:
                cls._instance = super(Wrapped, cls).__new__(cls)
            return cls._instance
    return Wrapped

@singleton
class MyClass:
    def __init__(self):
        print("Initializing MyClass")

# Test
a = MyClass()
b = MyClass()
print(a is b) # True - singleton behavior
```

What's Happening Here?

1. The `singleton` decorator takes the original class as input.
2. It creates a new inner class `Wrapped` that inherits from the original.
3. It overrides the `__new__` method to control instantiation (only one instance allowed).
4. It returns the modified class.

This approach implements the **Singleton pattern** without modifying the original class.

When to Use This Technique?

- When you want to **dynamically create modified versions** of classes.
- To **wrap class logic** with additional behavior like logging, validation, or access control.
- In design patterns like **Singleton**, **Proxy**, **Factory**, etc.